

CS 354 Fall 2025

Lab 9: A Device That Allows Access To Kernel Data Structures [20 pts]

Due: 12/05/2025 (Fri), 11:59 PM

Background

- Review how the remote file system uses pseudo-devices to allow a process to access a file.
- Review the format of the Configuration file that allows a user to declare new device types and new devices.

Introduction

Operating systems often use one facility to integrate access to multiple types of services. Unix-based systems, including Linux, have often used the file system. For example, in addition to integrating device access, Unix file system and file descriptors have been used to provide access to network communication, pseudo ttys, processes, message passing services, and access to virtual machines.

Unlike Unix, where everything is a file, everything in Xinu is a device. In this lab, you will create a special Xinu device that provides access to kernel data structures.

What You Will Learn

How a device is configured in Xinu. How device-independent I/O calls (open, read, write, and close) can be used to allow an application to access kernel data structures. How a set of pseudo-devices are used to allow concurrent access to a service.

The General Idea

You will design a Xinu device named `KERNEL_ACCESS` that allows a process to access Kernel data structures. For example, a process will be able to open your device to access the process table:

```
did32 desc;  
desc = open(KERNEL_ACCESS, "proctab", "r");
```

The call will return a descriptor for a pseudo-device. The process can use the descriptor by calling `read` to read successive entries in the table:

```
int32 result;  
struct procent buffer;  
result = read(desc, &buffer, sizeof(struct procent));
```

Each call will return one entry in the process table, copying the item from the process table entry to the specified buffer. After all entries have been read, the next call to `read` will return EOF.

Once the process finishes, it should close the pseudo-device.

```
close(desc);
```

Your device will support access to the process table ("proctab"), semaphore table ("semtab"), and device switch table ("devtab").

Steps You Will Take

0. Listen to the TAs describe the project and give hints.
1. The best way is to start modifying the **Configuration** file (**config/Configuration**) consisting of using the remote file system as an example and creating two new device types: `kad` (for the main Kernel access device) and `kapd` (for a Kernel access pseudo-device). Also define a single `kad` device named `KERNEL_ACCESS` and a set of pseudo-devices named `KACCESS0`, `KACCESS1`, `KACCESS2`, ...`KACCESS10`.
2. Create an include file, `kad.h`, that contains two structs and symbolic constants for specific values.

One struct will be needed for the `KERNEL_ACCESS` device. It must contain a list of

names for kernel items that can be accessed (e.g., "proctab", "semtab", "devtab"), and possibly a value for the next pseudo-device to examine when looking for a free entry.

The second struct will contain data items that will be used for a given pseudo-device. Among other things, the second struct will need fields that specify whether the device is currently free or allocated, and while the entry is allocated, an index that tells which entry in the kernel data structure to access on the next read operation, the size of the entry, and the maximum number of entries in the table.

Hint: look at the remote file system control block for ideas.

3. Build two initialization functions that Xinu will call at startup. Function `kadinit` (in `device/kaccess/kadinit.c`) will be used with your kad device and `kapdinit` (in `device/kaccess/kapdinit.c`) will be used with each pseudo-device. Note that the `kapdinit` program will be called with a pointer to the device switch table entry. You will need to extract the minor device number from the device switch table to know which entry in the control block array to use.

In the `kadinit` function define an external variable that has a type given by the first struct described above, and in the `kapdinit` function, define an external variable that is an array of the second struct described above, one entry in the array per pseudo-device.

4. Add prototypes for your functions to `include/prototypes.h` and add your initialization functions to the `Configuration` file.

5. Test your init functions to ensure that they correctly initialize the data structures.

6. Build a `kadopen` function (in `device/kaccess/kadopen.c`) for the `KERNEL_ACCESS` device that chooses a free pseudo-device, allocates it, and returns the descriptor. Look at the remote file system for an example. Check the mode specified by the third argument: only "r" (read) is acceptable.

7. Test your open function by calling

```
did32      desc;  
desc = open(KERNEL_ACCESS, "proctab", "r");
```

to see if the descriptor gives the device ID of one of your pseudo-devices. Test your open

function by making the above call repeatedly until your function returns SYSERR (i.e., until all pseudo-devices have been allocated and then one more open call was attempted).

8. Write a `kapdread` function (in `device/kaccess/kapdread.c`) that returns the next entry in whichever kernel table was specified in the `open` call. When a process calls `read`, the call will specify the buffer and the length of the buffer. If the length is zero, return SYSERR. Otherwise, copy up to length bytes from the entry in the kernel data structure into the caller's buffer, stopping at the actual size of the kernel data structure entry. Return the number of bytes copied into the buffer.

9. Test your `kapdread` function by opening the `KERNEL_ACCESS` device with "proctab" to obtain a descriptor (as in step 7 above). Then call `read` on the descriptor with a buffer large enough to store a `procent`:

```
byte      buffer[BUF_SIZE];
int32     result;
result =  read(desc, buffer, BUF_SIZE);
```

and verify that the result is exactly `min(BUF_SIZE, sizeof(struct procent))`.

10. Extend the previous test by repeatedly calling `read` until EOF is reached. Record how many times `read` returned a value before EOF and compare to `NPROC`.

11. Instead of merely checking the number of bytes read, perform a full byte-by-byte comparison between the bytes in the process table entry and the bytes in the buffer returned by `read` for each entry in the table. Start at location 0 in the process table, call `read` as above, and compare the bytes in the process table entry to the bytes in the buffer. Move to the next process table entry, call `read` again, and compare again for each entry in the process table.

12. Write additional functions for your driver, such as `kapdclose`. Look at the remote file system to see what functions it uses. Add each function to both `include/prototypes.h` and the `Configuration` file.

13. Test closing a descriptor, both after receiving an EOF and at other points (e.g., immediately after opening before any read calls, after 1 read call).

14. Create a `kapdseek` function (in `device/kaccess/kapdseek.c`) that allows one to seek to an arbitrary position in a table. For example, to read entry 2 in the process table

(i.e., the entry for process 2), one would make the following calls:

```
byte      buffer[BUF_SIZE];
int32     result;

desc = open(KERNEL_ACCESS, "proctab", "r");
seek(desc, 2);
result = read(desc, buffer, BUF_SIZE);
```

15. Test your seek function and make sure that the entry returned is indeed the same as entry 2 in the process table.

16. Extend your KERNEL_ACCESS device to support access to the semaphore table ("semtab") and device switch table ("devtab").

17. Test the extensions added in the previous step.

18. Test concurrent access to your KERNEL_ACCESS device. Create and resume four priority 10 processes. Have two of them open the KERNEL_ACCESS device for "devtab" and two of them open the KERNEL_ACCESS device for "proctab". Have all processes iterate by reading table entries until read returns EOF. To force concurrent execution, have each process call `yield()` after each call to `read`. Count the number of times read returns, and then use `kprintf` to print a summary before the process exits:

```
process XX for proctab found XX entries before EOF
process XX for proctab found XX entries before EOF
process XX for devtab  found XX entries before EOF
process XX for devtab  found XX entries before EOF
```

To start the processes, have the main process exit after creating and resuming the four processes.

19. Use the code you wrote for lab 5 that dumps values to test your KERNEL_ACCESS device. For each of the process table, semaphore table, and device table, open the KERNEL_ACCESS device for the table and iterate through entries by calling `read`. After each call to `read`, dump the table entry (i.e., print values of the fields as you did in lab 5).

Submit

General instructions:

When implementing code in the labs, please maintain separate versions/copies of code so that mistakes such as unintentional overwriting or deletion of code is prevented. This is in addition to the efficiency that such organization provides. You may use any number of version control systems such as GIT and RCS. Please make sure that your code is protected from public access. For example, when using GIT, use git that manages code locally instead of its on-line counterpart github. If you prefer not to use version control tools, you may just use manual copy to keep track of different versions required for development and testing. More vigilance and discipline may be required when doing so.

You must submit a directory named `lab9` that **contains all files, particularly the ones listed below**, and **make sure to “make clean” before submission and omit the cross_compiler folder**.

```
|--- xinu-bbb

|----- compile

|----- ...

|----- config

|----- Configuration

|----- ...

|----- DESCRIPTION

|----- device

|----- kaccess

|----- kadinit.c

|----- kapdinit.c

|----- kadopen.c

|----- kapdclose.c

|----- kapdread.c
```

```
|----- kapdseek.c
|----- include
|----- prototypes.h
|----- lib
|----- ...
|----- net
|----- ...
|----- shell
|----- ...
|----- system
|----- ...
```

Go to the directory where lab9 is a subdirectory.

For example, if /homes/alice/cs354/lab9 is your directory structure, go to /homes/alice/cs354.

Type the following command to submit the directory with `turnin`:

```
turnin -c cs354 -p lab9 lab9
```

You can check/list the submitted files using

```
turnin -c cs354 -p lab9 -v
```